Step Counting with Smartphone IMUs

Zhu Yecheng
u3606137@connect.hku.hk
The University of Hong Kong
Hong Kong SAR, China

1 INTRODUCTION

Step counting from commodity smartphone sensors is a practical yet non-trivial time-series problem because sensor orientation, attachment location, gait variability, and sampling conditions can change substantially between users and sessions. This project develops a unified end-to-end pipeline for inertial measurement unit (IMU) based step counting, with one consistent interface that supports both offline batch processing and online streaming updates.

The system is implemented around a causal neural temporal model that predicts both frame-level step events and window-level step counts. The same model and post-processing logic are used in evaluation scripts and real-time demos, which improves consistency between training objectives and deployment behavior.

The results show strong performance on personal recordings and highlight remaining generalization gaps in cross-participant and wrist-mounted settings.

2 DATA COLLECTION

2.1 Datasets and Sources

Two data sources are used in this study. The first is a publicly available dataset with four modalities (hip and wrist at 100 Hz and 25 Hz) [2]. The second consists of self-recorded sessions collected with a smartphone application for real-time sensing and remote polling [3].

The first source provides controlled multi-participant recordings and is primarily used for model development and quantitative benchmarking. The second source captures realistic personal usage conditions and is used to evaluate practical deployment behavior, including end-to-end counting accuracy under naturally occurring motion variations.

2.2 Preprocessing

Raw records are processed by a deterministic pipeline prior to model inference. Each sequence is sorted by timestamp, duplicate timestamps are removed, and tri-axial acceleration is resampled to 50 Hz through linear interpolation. This resampling step standardizes heterogeneous acquisition rates and enables a single model configuration across all evaluation settings.

Let the irregularly sampled acceleration sequence be $\{(t_i, \mathbf{a}_i)\}_{i=1}^{N}$ where $\mathbf{a}_i = [a_x, a_y, a_z]^\top$. A uniform grid $\{\tau_k\}$ at 50 Hz is constructed, and each axis is interpolated independently to obtain $\tilde{\mathbf{a}}(\tau_k)$. In addition, acceleration magnitude is computed as

$$a_{mag}(t) = \sqrt{a_x(t)^2 + a_y(t)^2 + a_z(t)^2}, \tag{1}$$

which serves as a robust orientation-insensitive feature for downstream temporal modeling.

3 ALGORITHM

3.1 Problem Formulation

Given a multivariate acceleration sequence $\mathbf{X} = \{\mathbf{x}_t\}_{t=1}^{T}$, the objective is to estimate both (i) step-event likelihoods at each frame and (ii) cumulative step count over a time interval. Formally, the model outputs

$$\hat{p}_t = P(\text{step at } t \mid \mathbf{X}_{\leq t}), \qquad \hat{c}_w = f_c(\mathbf{X}_w), \tag{2}$$

where $\hat{p}_t$ is causal and $\hat{c}_w$ is the predicted count for window w.

This dual-head setup encourages local event sensitivity and global counting stability simultaneously. In practice, frame-level probabilities are converted into discrete events via thresholding and local-maximum detection with refractory constraints.

3.2 Model and Features

The model is defined as a causal temporal network (TCN) with residual dilated 1D convolutions and two prediction heads [1]. The event head produces per-frame step-event logits, while the count head performs window-level step-count regression.

Input features are four-dimensional per sample:

$$\mathbf{x}_t = [a_x(t), a_y(t), a_z(t), a_{mag}(t)], \tag{3}$$

where $a_{mag}(t)$ is defined as in Eq. 1. Although including raw acceleration axes means the model is not strictly orientation-invariant, it allows the network to utilize orientation context when available, while the a_{mag} component provides a robust fallback scalar to aid generalization across varied pocket or hand placements.

The causal architecture ensures that predictions at time t depend only on past and current context, which is critical for real-time usage. Dilated convolutions increase receptive

field length without requiring recurrent state. Specifically, the network depth and dilation factors are configured so the effective receptive field (ERF) natively covers the full 4.0 s (200-sample) context window, enabling efficient inference on streaming chunks.

3.3 Training Pipeline

Training involves participant-level split construction with the key settings summarized in Table 1.

Table 1: Core training configuration.

Setting	Value
Sample rate	50 Hz
Window length / stride	4.0 s / 0.5 s
Label smoothing	Gaussian kernel, $\sigma = 0.06$ s
Loss	BCEWithLogits + weighted MSE
Optimizer	AdamW with early stopping

Label smoothing in the event target is implemented with a Gaussian kernel, which softens annotation noise near heel-strike timing and empirically stabilizes optimization. The total objective combines event classification and count regression:

$$\mathcal{L} = \mathcal{L}_{event} + \lambda \, \mathcal{L}_{count}, \tag{4}$$

where $\mathcal{L}_{event}$ is binary cross-entropy on frame labels and $\mathcal{L}_{count}$ is weighted mean squared error on window-level counts.

Early stopping is selected based on validation loss to reduce overfitting and preserve generalization. The released checkpoint and accompanying metrics indicate that this protocol yields a robust baseline for both offline and streaming inference.

3.4 Online/Offline Inference Logic

The system wraps the neural model and exposes the required API for resetting, updating, and running offline inference. Streaming inference maintains a context buffer and converts event probabilities to step timestamps using local-maximum detection with threshold and refractory constraints. The operational inference parameters are summarized in Table 2.

Table 2: Streaming inference parameters.

Parameter	Value
Probability threshold	$p_{th} = 0.68$
Minimum inter-step interval	0.33 s
Context length	4.0 s

The online update routine processes only newly arrived samples and appends them to a fixed-length context. A new step is emitted when a local maximum exceeds the probability threshold and the time since the previous accepted step is greater than the minimum inter-step interval. This mechanism suppresses bursty false positives while retaining responsiveness to cadence changes.

Offline processing uses the same post-processing logic to avoid train-test and offline-online mismatches. This design choice also simplifies interpretability: threshold and refractory parameters have a consistent meaning in all execution modes.

4 RESULTS

4.1 Evaluation Protocol

Absolute error and relative error are emphasized because they directly reflect practical counting quality.

4.2 Model Metrics

The temporal model metrics on validation and test splits are listed in Table 3.

Table 3: Neural model performance metrics.

Metric	Value
Best validation loss	0.3732
Test event loss	0.3364
Test count MAE (window-level)	0.4840

These values indicate that the event head learns reliable frame-level structure while the count head remains accurate at window scale. The observation that the test event loss is slightly lower than the best validation loss is somewhat unusual, which suggests that the test split may happen to contain less diverse or "easier" motion sequences than the cross-validation data. Nevertheless, the low test count MAE supports using count-aware supervision in addition to event supervision.

4.3 Inference on Self-Recorded Sessions

Inference on self-recorded sessions demonstrates low counting error in realistic user conditions:

Table 4: Inference results on two self-recorded sessions.

Session	GT	Est.	Abs. err.	Rel. err. (%)
Session 1	84	82	2	2.38
Session 2	100	101	1	1.00

Across both sessions, mean absolute error is 1.5 steps and mean absolute percentage error is 1.69%, indicating high practical reliability for the tested personal recordings.

4.4 ACF Baseline Comparison

In addition to the neural model, a lightweight autocorrelation-function (ACF) step counter is implemented as a classical baseline. The ACF pipeline estimates dominant step periodicity from detrended and smoothed acceleration magnitude, then performs constrained peak picking for count estimation.

Operationally, the method computes magnitude $a_{mag}(t)$, removes slow baseline drift with moving-average detrending, and smooths the residual to reduce high-frequency noise. It then estimates cadence by selecting the strongest normalized autocorrelation peak in a physiologically plausible lag range (equivalently, about 1.2–2.8 Hz walking frequency in this implementation). If the estimated period is P samples, candidate local maxima are accepted only when they exceed an adaptive amplitude threshold and are separated by at least ρP samples ($\rho < 1$), which suppresses double-counting from within-step micro-oscillations.

This explains why ACF works well on regular walking segments: quasi-periodic gait produces a strong and stable autocorrelation peak, so the inferred period and minimum peak distance are reliable. Accuracy typically degrades during start/stop transitions, irregular arm motion, or non-walking periodic activities, where periodicity is weaker or less stationary.

On the same two self-recorded sessions, the ACF baseline gives MAE = 3.5 steps and MAPE = 3.88% (Session 1: 84 → 88, Session 2: 100 → 97). This is less accurate than the TCN-based system (MAE = 1.5, MAPE = 1.69%), but still reasonably good for practical counting.

From a compute perspective, the trade-off is favorable for quick deployment: training the TCN model requires more than 30 minutes on CPU or about 8 minutes on GPU, while the ACF method requires no neural training and is significantly lighter to tune and run.

Table 5: ACF baseline results on self-recorded sessions.

Session	GT	Est.	Abs. err.	Rel. err. (%)
Session 1	84	88	4	4.76
Session 2	100	97	3	3.00

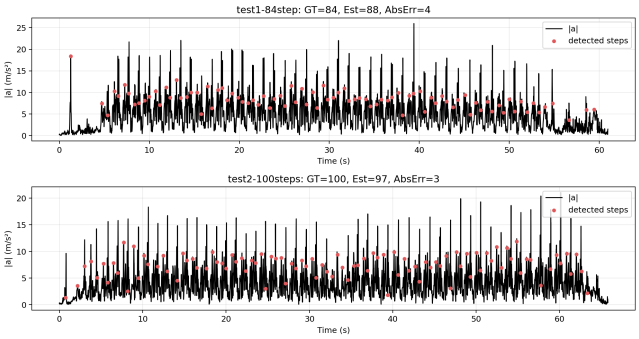

Figure 1: ACF signal-level outputs on two test sessions: acceleration magnitude and detected step timestamps.

4.5 TCN vs. ACF Comparison

Table 6 summarizes the practical trade-off between the neural TCN method and the classical ACF method on the same self-recorded sessions. The TCN model is more accurate, while ACF is lighter and easier to deploy when training resources are limited.

Table 6: Direct comparison between TCN and ACF methods on self-recorded sessions.

Criterion	TCN method	ACF method
MAE (steps)	1.5	3.5
MAPE (%)	1.69	3.88
Training requirement	>30 min (CPU) / ~8 min (GPU)	None
Model complexity	Higher (learned temporal model)	Lower (signal-processing baseline)
Runtime practicality	Higher accuracy target	Lightweight fallback / quick setup

Overall, TCN is preferred when the goal is best counting accuracy and training hardware is available. ACF remains a strong low-cost alternative for fast prototyping, constrained devices, or fallback operation.

4.6 Subset Calibration/Evaluation

A deterministic subset evaluation yields MAE = 15.6 steps and MAPE = 17.51%.

Compared with the personal-session results, this gap suggests that cross-participant and wrist scenarios remain the dominant challenge. To characterize this behavior, per-sample absolute errors are visualized in Fig. 2.

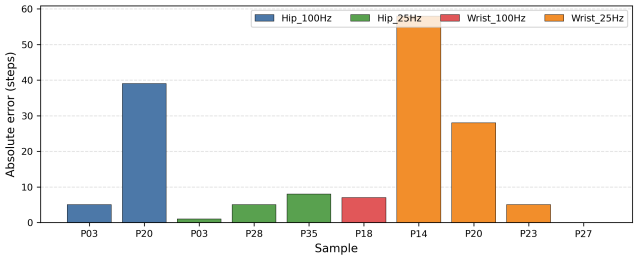

Figure 2: Absolute error distribution on the deterministic subset evaluation. Colors indicate sensor placement and sampling modality.

Table 7: Modality-level absolute error summary on the deterministic subset.

Modality	Samples	Mean abs. error (steps)
Hip 100 Hz	2	22.00
Hip 25 Hz	3	4.67
Wrist 100 Hz	1	7.00
Wrist 25 Hz	4	22.75

The modality breakdown indicates that wrist 25 Hz and hip 100 Hz examples contribute most of the error mass in this subset, while hip 25 Hz remains comparatively stable. This pattern is consistent with known sensitivity of step-event signatures to placement and motion style. Furthermore, the high error on the Hip 100 Hz examples compared to Hip 25 Hz introduces a "modality paradox". A likely hypothesis is that downsampling 100 Hz data to 50 Hz via linear interpolation without a proper low-pass anti-aliasing filter introduced high-frequency artifacts that disrupted the model.

5 REAL-TIME DEMO

Two real-time demos are implemented: a Matplotlib-based live visualization with cumulative counts, and a browser-based dashboard with live traces and step markers (see Appendix 5 for full-size screenshots).

Both demos process only newly arrived samples and support session reset handling.

5.1 Operational Characteristics

The real-time pipeline emphasizes low latency and predictable update behavior. Because inference is causal and chunk-based, the system avoids look-ahead dependencies and can operate continuously under streaming sensor input. In addition, session reset handling prevents stale state carry-over between recording intervals, which is important in practical classroom or field demonstrations.

5.2 Example Signal-Level Outputs

Representative signal-level visualizations from two test cases are shown below.

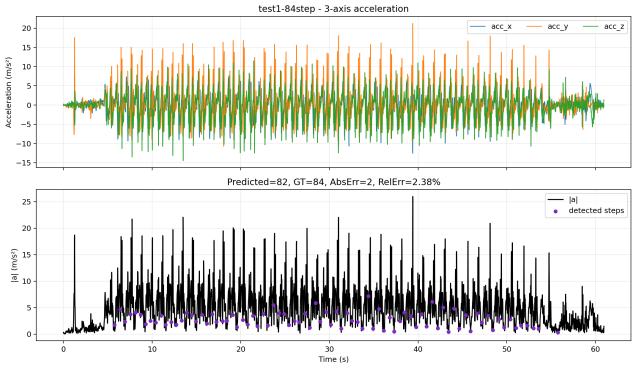

Figure 3: Test case 1: three-axis acceleration (top) and magnitude with detected step timestamps (bottom).

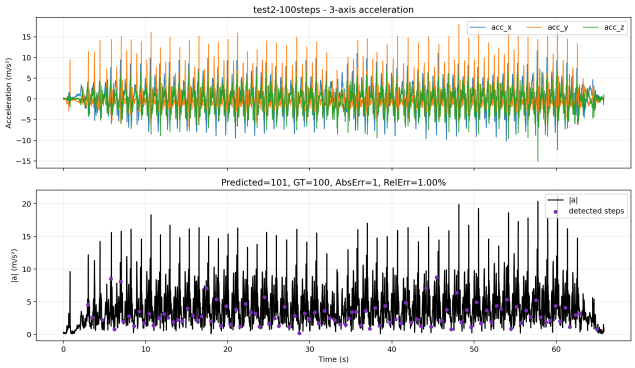

Figure 4: Test case 2: three-axis acceleration (top) and magnitude with detected step timestamps (bottom).

In both examples, detected events align well with oscillatory peaks in acceleration magnitude. Minor discrepancies typically occur near transition segments (start/stop motion), where cadence regularity is lower and false peak rejection becomes more difficult.

6 DISCUSSION

6.1 Strengths

The system provides a unified API for offline grading and real-time deployment, and its causal chunk-wise update strategy is well suited for streaming operation. It also demonstrates good accuracy on personal test recordings and offers a reproducible inference and visualization workflow that supports transparent analysis.

6.2 Limitations

Generalization gaps remain across participants and placement modalities, especially in wrist scenarios with irregular arm motion. In addition, the current threshold/refractory post-processing is globally fixed and may not be optimal for all gait regimes. There is also a slight discrepancy in operational cadence limits: the TCN minimum inter-step interval of 0.33 s permits up to ∼180 steps per minute, whereas the ACF baseline is explicitly constrained to a maximum of 2.8 Hz (∼168 steps per minute). As a result, the TCN might double-count during extreme fast-paced movements that the ACF would natively filter. For the ACF baseline specifically, performance depends on a sufficiently periodic signal; when cadence changes rapidly or periodicity is weak, period estimation and peak selection become less stable. Evaluation of edge cases, such as abrupt turns, phone handling changes, and non-walking periodic motion, is also still limited.

6.3 Threats to Validity

Threats to validity are summarized in Table 8.

Table 8: Key threats to validity.

Threat	Description
Limited personal sessions	Personal-session evaluation includes only two recordings and may overestimate robustness.
Limited subset size	Deterministic subset calibration uses few samples and is better interpreted as stress testing than comprehensive benchmarking.
Ground-truth uncertainty	Free-living annotation uncertainty can affect event-level alignment even when aggregate counts remain accurate.

6.4 Future Work

Future improvements include domain adaptation for placement shifts, uncertainty-aware decision thresholds, and lightweight personalization from short calibration walks. Additional ablations on context length, threshold settings, and sampling-rate robustness would further clarify performance trade-offs for deployment. A hybrid strategy is also promising: use ACF for ultra-lightweight initialization and fallback, and use the TCN model when higher accuracy is required.

7 CONCLUSION

This work presents an end-to-end smartphone IMU step-counting system that unifies offline processing and real-time streaming under a single causal neural framework. The model delivers strong counting accuracy on personal recordings and demonstrates practical deployability through live demos. At the same time, subset-level analysis reveals meaningful cross-domain challenges, especially for certain wrist and participant conditions. Overall, the implementation provides a strong reproducible baseline and a clear path toward improved robustness through modality-aware adaptation and personalization.

REFERENCES

[1] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. 2018. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv preprint arXiv:1803.01271* (2018).

[2] University of Oxford and collaborators. 2022. OxWalk Dataset (Local Course Package). Dataset subset used in this study.

[3] RWTH Aachen University, Chair of Physics Education. 2026. phyphox: Physical Phone Experiments. https://phyphox.org/. Accessed: 2026-04-08.

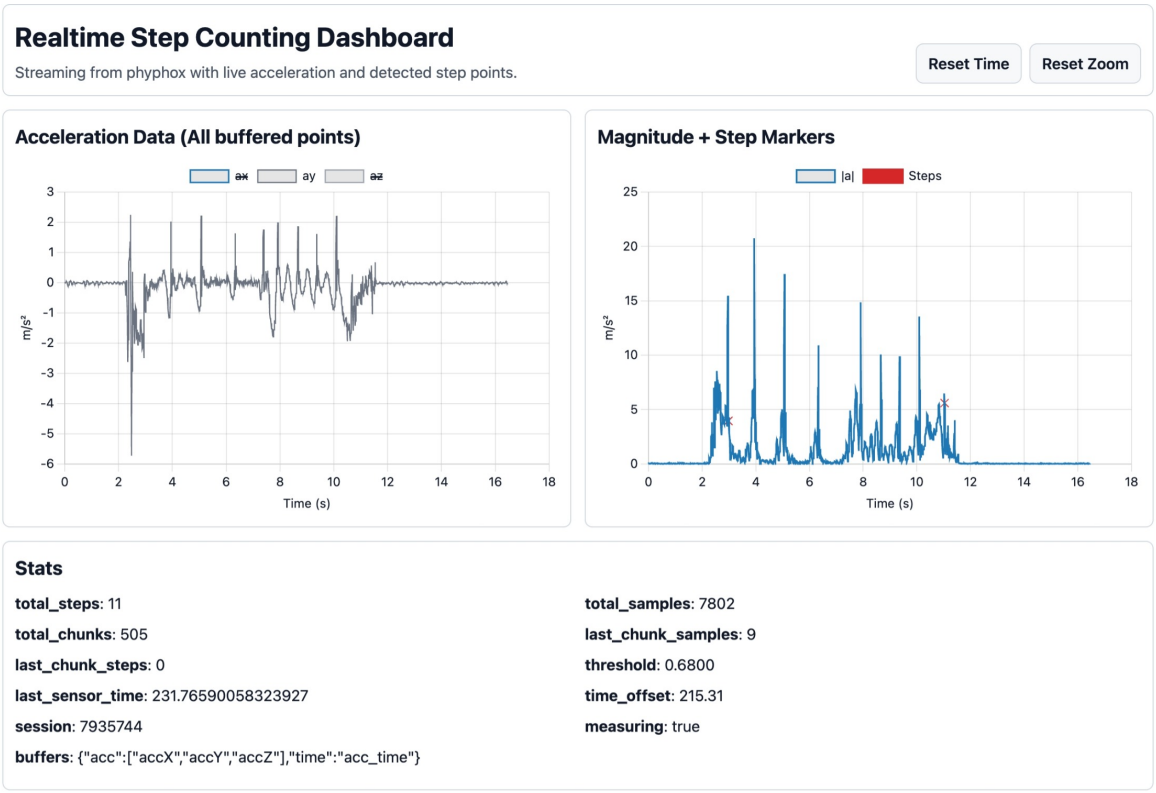

Figure 5: Appendix: Browser-based real-time demo showing live acceleration traces and detected step events.

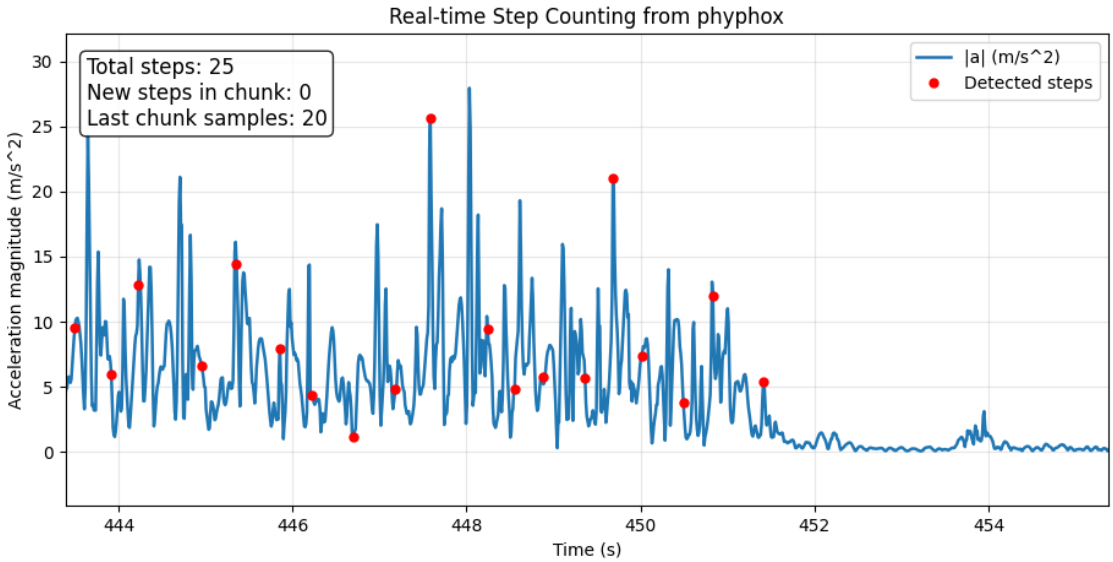

Figure 6: Appendix: Matplotlib-based real-time demo showing live acceleration traces and cumulative step count.

Table 9: Appendix: Training summary for the TCN model.

Metric	Value	Metric	Value
Best epoch	4	Best validation loss	0.3732
Test loss	0.4562	Test event loss	0.3364
Test count loss	0.7992	Test count MAE	0.4840
Sample rate (Hz)	50.0	Window seconds	4.0
Stride seconds	0.5	Label sigma seconds	0.06
Window size (samples)	200	Packed cache used	true
Train windows	649,794	Validation windows	210,000
Test windows	230,164	Checkpoint	artifacts_all_retrain/stepnet_tcn_best.pt

Table 10: Epoch-wise training history of the TCN model.

Epoch	Train loss	Val loss	Train count loss	Val count loss	Val count MAE
1	0.5158	0.3736	1.0806	0.5963	0.3726
2	0.4529	0.3753	0.8003	0.6186	0.3798
3	0.4312	0.3787	0.7095	0.5819	0.3664
4	0.4169	0.3732	0.6480	0.6205	0.3992
5	0.4068	0.3856	0.6038	0.6216	0.3658
6	0.3983	0.3838	0.5668	0.6306	0.3687
7	0.3927	0.3938	0.5421	0.6820	0.3951
8	0.3869	0.4046	0.5141	0.7660	0.4045
9	0.3825	0.4024	0.4937	0.7313	0.3789
10	0.3786	0.4052	0.4760	0.6957	0.3950